

在應用程式中加入動態色彩

來源：<https://codelabs.developers.google.com/codelabs/apply-dynamic-color?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您會將應用程式中的主題設定遷移至 Material 3，並在之後套用動態色彩。

目錄

1. [1. 事前準備](#)
2. [2. 範例應用程式總覽](#)
3. [3. 更新 Gradle 依附元件](#)
4. [4. 瞭解顏色主題和顏色角色](#)
5. [5. 使用 Material Design 主題設定建構工具建立 Material 3 主題](#)
6. [6. 使用 Material Design 主題設定建構工具匯出檔案](#)
7. [7. 新增動態色彩](#)
8. [8. 套用自訂主題疊加層](#)
9. [9. 為自訂屬性新增動態色彩](#)
10. [10. 摘要](#)

1. 事前準備

在本程式碼研究室中，您將更新入門應用程式 (小費計算機應用程式)，使用 Material Design 3 的新功能，根據使用者的桌布動態設定應用程式的使用者介面主題。以下是套用動態色彩的應用程式螢幕截圖。您也會瞭解其他情境，進一步掌控色彩的套用方式。

必要條件

開發人員應

- 熟悉 Android 的基本主題概念
- 輕鬆修改應用程式主題

課程內容

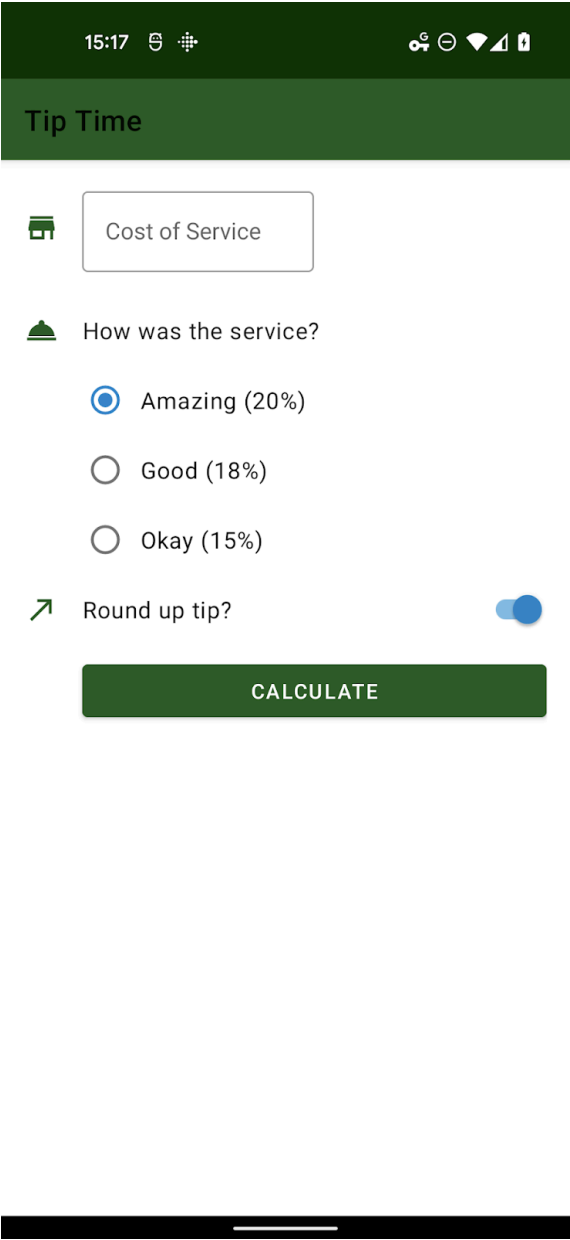
- 如何區分現有的 Material Components 和 Material 3 主題
- 如何將主題更新為 Material 3
- 如何使用我們的工具建立主題並套用
- 佈景主題屬性之間的關聯

軟硬體需求

- 已安裝 Android Studio 的電腦。
 - Tip Time 應用程式的程式碼。 <https://github.com/google-developer-training/android-basics-kotlin-tip-calculator-app-solution>
-

2. 範例應用程式總覽

Tip Time 應用程式是一款小費計算機應用程式，且提供可自訂小費的選項。這是 Android Kotlin 基本概念訓練課程的其中一個範例應用程式。



3. 更新 Gradle 依附元件

更新實際主題並套用動態色彩前，您需要在應用程式的 build.gradle 檔案中進行幾項變更。

在依附元件區段中，確認 Material 程式庫為 **1.5.0-alpha04** 以上版本：

□□□

```
dependencies {  
    // ...  
    implementation 'com.google.android.material:material:<version>'  
}
```

在 android 區段中，變更 compileSdkVersion 和 targetSdkVersion

31 以上：

□□□

```
android {  
    compileSdkVersion 31  
    // ...  
  
    defaultConfig {  
        // ...  
        targetSdkVersion 31  
    }  
}
```

這些操作說明假設應用程式具有相對較新的依附元件。如果應用程式尚未使用 Material 或較舊版本，請參閱 [Android 適用的 Material Design 元件](#) 的「開始使用」說明文件。

在您建立主題的任何位置，將 `Theme.MaterialComponents.*` 的參照變更為 `Theme.Material3.*`。部分樣式在 Material3 命名空間中尚未有新樣式，但只要父項主題更新為 `Theme.Material3.*`，大多數元件仍會繼承新樣式。如下所示，按鈕現在採用新的圓角主題。

在下方的主題檔案中，唯一變更的項目是父項主題。我們稍後會完全取代這個主題。部分顏色屬性已淘汰，我們建立的部分自訂樣式現在是 Material 3 的標準樣式，但我們希望您能

themes.xml

```
<style name="Theme.TipTime" parent="Theme.Material3.Light">
  <!-- Primary brand color. -->
  <item name="colorPrimary">@color/green</item>
  <item name="colorPrimaryVariant">@color/green_dark</item>
  <item name="colorOnPrimary">@color/white</item>
  <!-- Secondary brand color. -->
  <item name="colorSecondary">@color/blue</item>
  <item name="colorSecondaryVariant">@color/blue_dark</item>
  <item name="colorOnSecondary">@color/black</item>
  <!-- Status bar color. -->
  <item name="android:statusBarColor" tools:targetApi="l">?attr/colorPrimaryVariant</item>
  <!-- For text input fields -->
  <item
name="textInputStyle">@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox</item>
  <!-- For radio buttons -->
  <item name="radioButtonStyle">@style/Widget.TipTime.CompoundButton.RadioButton</item>
  <!-- For switches -->
  <item name="switchStyle">@style/Widget.TipTime.CompoundButton.Switch</item>
</style>
```

15:19



Tip Time



Cost of Service



How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)



Round up tip?



Calculate

4. 瞭解顏色主題和顏色角色

Material 3 色彩系統會以有條不紊的方式，將色彩套用至 UI。Theme.AppCompat 的許多屬性仍在使用中。不過，Theme.MaterialComponents.* 中新增了更多屬性，Theme.Material3.* 中甚至新增了更多屬性，因此請務必檢查應用程式的所有畫面，確保沒有任何未實作的屬性從基本主題滲透進來。

瞭解顏色扮演的角色

Material 3 主題中與顏色相關的屬性超過二十個。一開始可能覺得很困難，但其實只要搭配幾種主要顏色和 4 到 5 種顏色角色，就能建立衍生色。

這些顏色群組包括：

- 主要：應用程式的主要顏色
- 次要：應用程式的次要顏色
- 第三色，與原色和次要色互補
- 錯誤，用於錯誤文字和對話方塊
- 背景
- Surface、SurfaceVariant、Surface Inverse

原色、次要色、第三色和錯誤的顏色角色如下：

<base color>	底色
on<base color>	顯示在基底顏色上的圖示和文字顏色
<base color>Container	衍生自基本顏色，用於按鈕、對話方塊等
on<base color>Container	容器中圖示和文字的顏色

舉例來說，Material 3 中的預設樣式懸浮動作按鈕會使用 Primary 做為基本顏色，因此按鈕的背景顏色為 primaryContainer，內容顏色為 onPrimaryContainer。

手動自訂主題時，至少應確認變更的每個基底顏色 on<base color> 屬性仍可辨識。

最佳做法是同時調整顏色群組中的所有角色，確保從基礎到應用程式都沒有任何構件。

背景和表面基底顏色通常有兩種角色，分別是基底顏色本身，以及顯示在基底顏色上的圖示或文字的 on<base color>。

5. 使用 Material Design 主題設定建構工具建立 Material 3 主題

透過 Material Design 主題設定建構工具，您可以輕鬆建構自訂色彩配置、使用內建的程式碼匯出功能遷移至 M3 色彩系統，以及善用動態色彩。詳情請參閱 material.io/material-theme-builder

「小費計算機」應用程式主題包含多種元件樣式，但大多數樣式都是 Material 3 主題中的預設樣式。我們只需要注意主要和次要這兩種主要顏色。

這對應到綠色主要顏色 (#1B5E20) 和藍色次要顏色 (#0288D1)。

您可以將這些顏色輸入 Material Design 主題設定建構工具，然後匯出完整主題 (假設連結至其他地方的完整總覽)。

請注意，您輸入的顏色可能會因應顏色生成演算法而調整色調，確保顏色互補且可讀。

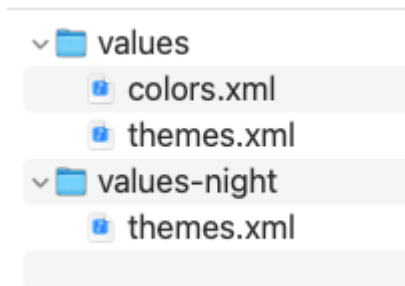
以下是輸入自訂顏色時產生的一組值。

Light				Dark			
Primary	On Primary	Primary Container	On Primary Container	Primary	On Primary	Primary Container	On Primary Container
Primary40	Primary100	Primary90	Primary10	Primary80	Primary20	Primary30	Primary90
Secondary	On Secondary	Secondary Container	On Secondary Container	Secondary	On Secondary	Secondary Container	On Secondary Container
Secondary40	Secondary100	Secondary90	Secondary10	Secondary80	Secondary20	Secondary30	Secondary90
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container	Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Tertiary40	Tertiary100	Tertiary90	Tertiary10	Tertiary80	Tertiary20	Tertiary30	Tertiary90
Error	On Error	Error Container	On Error Container	Error	On Error	Error Container	On Error Container
Error40	Error100	Error90	Error10	Error80	Error20	Error30	Error90

步驟 6 · 約 0 分鐘

6. 使用 Material Design 主題設定建構工具匯出檔案

匯出封存檔包含值和 values-night/ 目錄，以及各自的 themes.xml 檔案，分別對應淺色和深色主題。所有顏色都會在 values/colors.xml 中定義。



檔案可以照原樣複製，但您必須變更 AndroidManifest.xml 中的主題名稱，或變更主題檔案中的名稱，確保兩者一致。工具的預設名稱為 AppTheme。

重新啟動應用程式，畫面幾乎完全相同。其中一項顯著的變更就是「切換」和「單選按鈕」，這兩者的選取狀態現在會以主要顏色而非次要顏色的色調為主題。如果是較大的應用程式，您可能需要重新檢視某些設計。

15:20

Tip Time

 Cost of Service

 How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)

 Round up tip? ☒

Calculate

注意：您可能會收到有關 AndroidManifest.xml 檔案的錯誤訊息，指出 Android 12 應用程式需要為活動指定 `android:exported` 的明確值。

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.tiptime">

    <application ...>
        <activity android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

步驟 7 · 約 0 分鐘

7. 新增動態色彩

使用適當的 Material 3 主題後，我們只要稍做調整，就能讓 UI 呈現動態效果。

動態色彩 API 可讓您將動態色彩套用至所有 Activity

應用程式、個別活動，或個別檢視畫面或片段。適用於

這個應用程式，我們將在全球套用動態色彩。

建立 Application 類別檔案

```
class TipTimeApplication: Application() {  
    override fun onCreate() {  
        // Apply dynamic color  
        DynamicColors.applyToActivitiesIfAvailable(this)  
    }  
}
```

我們需要在 Android 資訊清單中參照這個新建立的檔案：

AndroidManifest.xml

```
< application android:name=".TipTimeApplication"  
<!-- Other existing attributes -->  
  
</application >
```

在 Android 12 以上的系統中，系統會檢查使用者的桌布 (預設配置)，產生數種色調調色盤。這些調色盤的值是用來建立 `ThemeOverlay`。

`DynamicColors` 類別會註冊 `ActivityLifecycleCallbacks`，攔截 `ActivityPreCreated` 以套用系統建立的動態主題疊加層，或您提供的疊加層。

Tip Time



Cost of Service



How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)



Round up tip?



Calculate

8. 套用自訂主題疊加層

我們的工具可以匯出主題疊加層，但如果您要覆寫的屬性數量不多，也可以手動建立。

主題疊加層應搭配其他主題使用，且只提供要在基本主題上變更的值。

假設我們因為某些原因 (例如品牌宣傳) 需要使用紅色調做為主要顏色，我們可以使用下列檔案和屬性來完成這項作業。

colors.xml

```
<resources>
    <color name="overlay_light_primary">#9C4146</color>
    <color name="overlay_light_onPrimary">#FFFFFF</color>
    <color name="overlay_light_primaryContainer">#FFDADB</color>
    <color name="overlay_light_onPrimaryContainer">#400008</color>
</resources>
```

themes_overlays.xml

```
<style name="AppTheme.Overlay" parent="ThemeOverlay.Material3.DynamicColors.Light">
    <item name="colorPrimary">@color/overlay_light_primary</item>
    <item name="colorOnPrimary">@color/overlay_light_onPrimary</item>
    <item name="colorPrimaryContainer">@color/overlay_light_primaryContainer</item>
    <item name="colorOnPrimaryContainer">@color/overlay_light_onPrimaryContainer</item>
</style>
```

對於上述程式碼，Android 12 會套用動態淺色主題，並將變更內容疊加在該主題上。或者，您也可以使用任何有效的 ThemeOverlay 做為父項，包括下列任一項：

ThemeOverlay.Material3.Light

ThemeOverlay.Material3.Dark

ThemeOverlay.Material3.DayNight ThemeOverlay.Material3.DynamicColors.Dark

ThemeOverlay.Material3.DynamicColors.DayNight

如要使用這個主題疊加層，而非 Material 預設主題，請將呼叫變更為

DynamicColors.applyToActivitiesIfAvailable：

```
DynamicColors.applyToActivitiesIfAvailable(this, R.style.AppTheme_Overlay)
```

Tip Time



Cost of Service



How was the service?



Amazing (20%)



Good (18%)



Okay (15%)



Round up tip?



Calculate

9. 為自訂屬性新增動態色彩

到目前為止，我們已覆寫 Material 3 主題中已有的屬性。在動態色彩中，我們可能有一或多個需要分配的自訂屬性。

應用程式選擇啟用動態色彩後，即可存取 5 個色調調色盤，包括 3 個強調色調色盤和 2 個中性色調色盤，大致具有下列角色：

system_accent1	主要色調
system_accent2	次要色調
system_accent3	三次色調
system_neutral1	中性背景和表面
system_neutral2	中性表面和外框

每個調色盤都有多個色階，從白色到黑色：0、10、50、100、200、300、400、500、600、700、800、900、1000。

設計動態色彩的 UI 時，您應減少考慮特定顏色，多考慮該元件的色調和亮度與設計系統中其他元件的關係。

假設您希望圖示使用次要強調色調色盤，並在 attrs.xml 中新增屬性，為圖示加上色調，如下所示：

attrs.xml

```
<resources>
    <attr name="iconColor" format="color" />
</resources>
```

主題疊加層可能如下所示：

```
<style name="AppTheme.Overlay" parent="ThemeOverlay.Material3.DynamicColors.DayNight">
    <item name="iconColor">@android:color/system_accent2_600</item>
</style>
```

重新安裝應用程式並變更桌布後，應用程式就會採用次要色調。

Tip Time



Cost of Service



How was the service?

☒ Amazing (20%)

☐ Good (18%)

☐ Okay (15%)



Round up tip?



Calculate

15:38





Tip Time



Cost of Service



How was the service?

☒

Amazing (20%)

☐

Good (18%)

☐

Okay (15%)



Round up tip?

☐

Calculate

這些調色盤專為 Android 12 (API 31) 設計，因此除非應用程式的最低 SDK 設為 31 以上，否則您必須將相關檔案放在含有 `-v31` 後置字串的資料夾中。

10. 摘要

在本程式碼研究室中，您已完成下列事項：

- 新增依附元件，將主題升級至 Material 3。
 - 瞭解新的顏色群組和角色。
 - 瞭解如何從靜態主題遷移至動態色彩。
 - 瞭解如何使用主題疊加層，以及如何為自訂主題屬性使用動態顏色。
-